

微服务拆分原则

- 前提
 - 业务理解一致，统一语义
 - 职责边界清晰
 - 持续迭代和改进的过程
- 方法
 - 领域模型
 - 中间件
 - 性能
 - 团队和人性
 - 符合团队结构
 - 维护成本
 - 每个人最好有一个重要的微服务

- 原则
 - IDEALS原则
 - 接口分离（Interface segregation）
 - 让不同类型的客户端只能看到自己所对应的接口（服务契约）
 - 移动端
 - 后台
 - 小程序
 - 第三方
 - 可部署性（deployability）
 - 配置运行时基础设施，包括容器、pods、集群、持久性、安全性和网络。
 - 微服务的扩缩容，或者将他们从一个运行时环境迁移到另一个运行时环境。
 - 加速提交+构建+测试+部署过程
 - 减少版本升级时的停机时间
 - 同步相关软件的版本更改
 - 监控微服务运行状况，以快速识别和修复故障。
 - 事件驱动（event-driven），提高了可伸缩性和吞吐量
 - HTTP调用（REST服务）
 - 使用特定于平台的组件技术进行类RPC调用，如gRPC或GraphQL
 - 通过消息中间件处理异步消息
 - 可用性胜于一致性（Availability over Consistency）
 - 最终一致性的可用性
 - 松耦合（Loose coupling）
 - 耦合是指两个软件元素之间相互依赖的程度
 - 单一责任（single responsibility）
 - 其它
 - 边界上下文
 - 微服务的粒度不能大于领域驱动里的 Bounded Context（具体是什么大家自行 Google），也就是一个业务域
 - 单一职责，高内聚，低耦合
 - 把因为相同原因变化的合在一起（内聚），把不同原因变化的分开（解耦）
 - 事务和一致性
 - 对于两个重度依赖的功能，需要完成一个事务和要保证强一致性的，最好不要拆开，要放在一起。
 - 跟组织架构匹配
 - 把同一个团队的东西放在一起，不同团队的分开
 - “一切以Service第一”为系统架构的公司

- AI给的答案（业自领团重演粒）
 - 按业务能力拆分
 - 这是微服务拆分的最基本原则之一，意味着根据业务功能的不同将应用程序拆分成多个服务
 - 领域驱动设计（DDD）
 - 领域驱动设计是一种软件开发方法，强调基于业务领域的逻辑来设计软件。在微服务架构中，可以利用DDD的概念来识别服务边界，将软件拆分成代表领域模型的微服务
 - 分离读写操作
 - 根据CQRS（Command Query Responsibility Segregation，命令查询责任分离）原则，可以将负责处理数据写入操作的服务与负责读取数据的服务分开。这有助于优化性能和扩展性，因为读写操作往往有不同的资源和性能需求
 - 服务自治原则
 - 每个微服务应该是自治的，拥有自己的数据库和数据存储，避免与其他服务共享数据库。这有助于减少服务间的耦合，提高服务的独立性和可维护性
 - 按团队组织结构拆分
 - Conway定律表明，系统设计往往会复制组织的沟通结构。按照团队的组织结构来拆分服务，可以确保每个服务的开发和维护由一个独立的团队负责，从而提高效率和减少沟通成本
 - 可重用性原则
 - 在拆分微服务时，应考虑服务的可重用性。通用的功能或组件应该设计成可由多个服务共享的微服务，以减少代码重复和提高开发效率
 - 演进式设计原则
 - 微服务架构应支持逐步演进和迭代。初始拆分不必完美，应随着时间的推移根据实际需要进行调整和优化
 - 服务粒度平衡
 - 在微服务拆分时需要平衡服务的粒度。过大的服务可能导致单体应用的问题，如难以维护和部署；而过小的服务可能会导致管理和通信的开销增大。找到合适的服务大小是关键

- 伸延
 - 面向对象的设计（SOLID原则）
 - 单一责任原则（Single responsibility principle）
 - 开闭原则（Open/closed principle）
 - 对扩展开放，对修改关闭
 - 里氏代换原则（Liskov substitution principle）
 - 任何基类可以出现的地方，子类一定可以出现。LSP是继承复用的基石，只有当衍生类可以替换掉基类，软件单位的功能不受到影响时，基类才能真正被复用，而衍生类也能够在基类的基础上增加新的行为
 - 接口分离原则（Interface segregation principle）
 - 接口分离原则描述为“客户类不应被迫依赖那些它们不需要的接口。多个与特定客户类有关的接口比采用一个通用的接口要好
 - 依赖倒置原则（Dependency inversion principle）
 - 程序要依赖于抽象接口，不要依赖于具体实现。